Instructions: use the online assignment administration platform to indicate which problems you have completed *and are able to present*. Create one short plain text file (`.txt` or `.md`) **per problem** in which you discuss your solution. Name these files `N.{txt, md}`, where `N` is the problem number in this list. Upload your solutions to our Moodle course.

1. Create a file that has last been modified 100 years ago by learning about the `touch` command. Can you also create a file that says its last modification date is from the year the University of Graz has been founded (1585)?

2. Create three empty files named `file1.txt`, `file2.txt`, `file3.txt` in your current directory. Then, list all files in the directory sorted by their last modification time (oldest first, newest last).

3. Use `curl` or `wget` to download the text file that is hosted at https://gist.githubusercontent.com/behackl/6620ab9cdfcef9587aed8ba605cf8e10/raw/a6afecfd6256e8778f397fe1310c39be78ace7dd/bee-movie-script.txt, which contains the script of the *Bee Movie*. Use `grep` to find (a) all lines starting with *Bee* (including instances like `Bees make too much of it.` and `bee law number one,`) and (b) all lines containing the word `bee` (regardless of capitalization). How many such lines are there, respectively?

4. Complete the free demo part of https://vim-adventures.com/. What is in the chest in the maze?

5. The emergency manual of the spaceship *SS Probably Fine* needs a little maintenance. Download the following file and edit it using a **terminal editor of your choice** (e.g. Vim, Neovim, Nano, or Emacs):

   https://imsc.uni-graz.at/hackl/teaching/spaceship-manual.txt

   - Correct every occurrence of `teh` to `the` using the editor's search-and-replace feature.
   - Find the emergency phone number and change it from `000-000-0041` to `000-000-0042`.
   - Remove one copy of the duplicated crew reminder.
   - Replace the placeholder under *Captain's note* with an instruction of your own.
   - Save the file, exit the editor, and reopen it to verify your changes.

   Include the edited manual with your solution and briefly note which editor and commands or shortcuts you used. If you need an introduction, try `vimtutor` (if installed), Neovim's `:Tutor`, or a beginner tutorial for your chosen editor.

6. The file `webserver.log` contains the following content:

   ```
   203.0.113.24 - - [25/Mar/2024:10:15:32 +0000] "GET /index.html HTTP/1.1" 200 2326
   198.51.100.10 - - [25/Mar/2024:10:15:33 +0000] "GET /images/logo.png HTTP/1.1" 200 5432
   203.0.113.24 - - [25/Mar/2024:10:15:34 +0000] "GET /css/style.css HTTP/1.1" 200 1054
   192.0.2.42 - - [25/Mar/2024:10:15:35 +0000] "POST /login HTTP/1.1" 302 0
   203.0.113.24 - - [25/Mar/2024:10:15:36 +0000] "GET /about.html HTTP/1.1" 200 3571
   192.0.2.42 - - [25/Mar/2024:10:15:37 +0000] "GET /dashboard HTTP/1.1" 200 5123
   198.51.100.10 - - [25/Mar/2024:10:15:38 +0000] "GET /products/item1234 HTTP/1.1" 404 1762
   203.0.113.24 - - [25/Mar/2024:10:15:39 +0000] "POST /contact HTTP/1.1" 200 982
   ```

   Use the programs `cut`, `sort`, `uniq`, `wc` to determine the number of different visitor IP addresses in this logfile. If you are satisfied, test your solution against the real sample logfile from here: https://raw.githubusercontent.com/elastic/examples/refs/heads/master/Common%20Data%20Formats/apache_logs/apache_logs.

7. At https://www.gutenberg.org/cache/epub/84/pg84.txt you can find a copy of the book *Frankenstein; Or, The Modern Prometheus*.
   - Determine the word count and the total number of **text characters** (letters, digits, punctuation, spaces, and line breaks) in the downloaded file.
   - Print all lines containing the word *Frankenstein* (including a context of 1 line before and after).

8. The department's debugging duck has disappeared! The investigation records are available in a compressed archive:

   https://imsc.uni-graz.at/hackl/teaching/duck-investigation.tar.gz

   - Use `curl` or `wget` to download the archive.
   - Learn about `tar` and list the archive's contents **without extracting it**.
   - Extract the archive into a new, empty directory.
   - Use `find` to locate `confession.txt`. Read it: who took the duck, and why?
   - Create a new compressed archive named `duck-investigation-repacked.tar.gz` containing the extracted `duck-investigation` directory.

   Record your commands and the answer in your solution; you do not need to upload the repacked archive.

9. Learn about the terminal multiplexer `tmux` by following the short tutorial at https://hamvocke.com/blog/a-quick-and-easy-guide-to-tmux/. Then find out why `tmux` is so useful in practice: start a session, run something long-running in it (e.g., `top`), detach, close your terminal window *completely*, open a new one, and reattach to your session. Is your program still running? Which commands did you use to list and reattach sessions?

10. The department's coffee machine runs a small program, `coffeed`, which writes everything it does to a log file. Download the program from

    https://imsc.uni-graz.at/hackl/teaching/coffeed.sh

    and start it in the background via `bash coffeed.sh &`; it logs to `coffeed.log` in the current directory. *(Tip: a `tmux` session with two panes is handy here.)*
    - Open the log with `less` (press `h` inside `less` for help). Jump to the end, back to the beginning, and to line 2718: who got which drink there?
    - Search for the `ERROR`: what went wrong, and when? Starting from there, search *backwards* for the closest `WARNING`.
    - Use `&` inside `less` to show only the `WARNING` lines. How many are there?
    - The machine is still brewing: press `F` to follow the log while new lines are written, stop following with Ctrl + c, and quit with `q`.
    - Find the process ID of `coffeed` and stop it *politely* using `kill`. What are the machine's final words? Would you also have seen them with `kill -9`? Why (not)?

11. Mission control never sends the crew of the *SS Probably Fine* complete files, only *patches* – that way, the crew can see exactly what changed before accepting it. Download the current pre-launch checklist and mission control's latest patch:

    https://imsc.uni-graz.at/hackl/teaching/checklist.txt
    https://imsc.uni-graz.at/hackl/teaching/mission-control.patch

    - Read the patch **before** applying it. What do lines starting with `-`, `+`, and a space mean? What do the numbers in `@@ -17,10 +17,10 @@` tell you?
    - Apply the patch using `patch`. What do the crew have to do in step 21 now? What happens if you try to apply the same patch a second time?
    - Mission control missed a rather important problem in the *Engines* section. Keep a copy of the patched checklist, fix the problem with your editor, and use `diff -u` to create a patch `reply.patch` containing your fix.
    - Check that your patch applies cleanly to a fresh copy of the patched checklist.

    Include `reply.patch` and the commands you used in your solution.

12. Write a small bash script `greeting.sh` that greets its user personally.
    - When called with an argument, e.g. `./greeting.sh Benjamin`, it should print `Hello, Benjamin!`.
    - When called without arguments, it should first ask for the name using `read`.
    - Running it as `./greeting.sh` should use `bash` to interpret the script: which line at the top of the file takes care of that?
    - What do you need to do before `./greeting.sh` works? Why does `bash greeting.sh` work even without that?
    - What happens for `./greeting.sh Ada Lovelace` and for `./greeting.sh "Ada Lovelace"`? Explain the difference.

    Hints: `$1` contains the first argument, `$#` the number of arguments. The scripting slides of Chapter 1 contain an example of an `if`-statement; `help read` and `help if` print documentation for these Bash builtins. *Optional:* make your script callable as just `greeting` from any directory.

13. The code snippet `:(){ :|:& };:` defines a bash function and calls it; the code is, however, written in a rather obfuscated way. What does it do? (Try running it, if you are brave.)

14. Read the first chapter[1] of the book Software Engineering at Google and prepare to discuss the content in the exercise session.

15. Learn about the command `ssh-keygen` and create a new SSH key pair using the option `-t ed25519` (what does it do?). Add the public key (usually located at `~/.ssh/id_ed25519.pub`) to your GitHub account (or any other Git hosting service of your choice if you don't want to use GitHub). Check that everything works by running `ssh -T git@github.com` (or the equivalent for your hosting service). What are you asked when connecting for the first time, and why? What does the reply tell you?

16. Read Chapter 16 (*Version Control and Branch Management*)[2] of Software Engineering at Google, as well as Anthropic's recent blog post on their in-house version control system *Scion*:

    https://www.anthropic.com/engineering/redesigning-source-control

    Prepare to discuss both texts in the exercise session, for example:
    - What is the difference between a centralized and a distributed version control system? Which one is Git, and what does Google use?
    - Why does Google avoid long-lived development branches?
    - Why did Git become a problem for Anthropic? Name one thing Scion does differently than Git.
    - Would you need anything like Scion for your project in this course? Why (not)?

17. Complete the chapters *Introduction Sequence* and *Ramping Up* (tab *Main*), as well as the first four levels of *Push & Pull – Git Remotes!* (tab *Remote*) of the interactive tutorial at https://learngitbranching.js.org/. Take notes of your solutions. Use the level *Reversing Changes in Git* to explain: when should you use `git reset`, and when `git revert`?

18. Time for your first real repository: the crew of the *SS Probably Fine* is planning its next mission.
    - Create a repository `mission-planning` with a file `crew.txt` listing the crew, one member per line (Dr. Honk insists on being on it). Compare the output of `git status` before and after `git add`, then commit using a Conventional Commit message.

[1] https://abseil.io/resources/swe-book/html/ch01.html
[2] https://abseil.io/resources/swe-book/html/ch16.html

- Make some changes to `crew.txt`. Compare `git diff` and `git diff --staged` before and after running `git add`, then commit again.
- The onboard computer keeps writing a `launch.log`, and the airlock code is stored in `.env`. Create both files, then write a `.gitignore` such that `git status` does not show them anymore, and commit it.
- Tag your latest commit as `v1.0`. Then learn about *aliases* in Git (https://git-scm.com/book/en/v2/Git-Basics-Git-Aliases) and set up the alias `git lg` for

```
log --graph --abbrev-commit --date=relative
    --pretty=format:'%Cred%h%Creset -%C(yellow)%d%Creset %s %Cgreen(%cr) %C(bold blue)[%an]%Creset'
```

  Include the output of `git lg` in your solution.

19. The computer algebra system SymPy has a history of almost 20 years. Clone it via

```
git clone --filter=blob:none https://github.com/sympy/sympy.git
```

- What does `--filter=blob:none` do, and why is it a good idea here? How large is your `.git` directory (`du -sh .git`)?
- Use `git log` to find out how often `sympy/abc.py` has been modified, who created it, and when.
- The file defines the Greek letter `lamda`, without a *b*. Use `git blame` to find out who last changed that line. Then use the pickaxe (`git log -S`) to find the commit that first added `lamda` to this file. *(And why is it even spelled `lamda`?)*

20. Mission control keeps the official log of the *SS Probably Fine* at

    https://imsc.uni-graz.at/git/behackl/ss-probably-fine-ships-log.git

    However, a log that exists only once is a log waiting to be lost.
    - Clone the log, add an entry of your own to `log.md`, and commit it. Try to `git push`: what happens, and why?
    - *Earth:* create an empty repository (no README, no license) on GitHub, GitLab, or Codeberg, add it as a remote named `earth` using its SSH URL (cf. Exercise 15), and push `main`.
    - *Cargo hold:* create a bare repository outside of your clone (`git init --bare ../cargo-hold.git`), add it as a remote named `backup`, and push.

    What does `git remote -v` show? Where does a plain `git push` go now, and why?

21. Create a new repository and commit a file `fruit.txt` containing some of your favorite fruits (one per line). Add a few more fruits in a second commit. *Before* running each of the following commands, predict what it does to your working directory, index, and history. Then run it and check your prediction with `git status`, `git log --oneline`, and `cat fruit.txt`:
    - `git revert HEAD`
    - `git reset --hard HEAD~1`
    - `git restore --source=HEAD~1 fruit.txt`
    - `git stash`
    - `git stash pop`

    The commit created by `git revert` is gone now. Or is it? Use `git reflog` to rescue it into a new branch `rescued`. Which reflog entry did `git stash` leave behind?